

José Antonio Vacas Martínez

Aprende a programar Python



Logo Python

Versión 0.99.2



Licencia CC by SA by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Trabajando con ficheros

A lo largo de este tema vamos a aprender a trabajar con ficheros, lo que nos va a permitir almacenar información de manera permanente y recuperarla posteriormente.

Como práctica, haremos un programa que es capaz de contar las líneas, palabras y caracteres de todos los ficheros que tenemos en un directorio. También haremos que nuestro programa de las 20 Preguntas pueda guardar su conocimiento para partidas posteriores.

Administración de ficheros

En este capítulo vamos a aprender a gestionar ficheros, usando los módulos *os*, *pathlib* y *shutils* de Python..

Por gestionar entendemos el encontrarlos en las carpetas, ver los que tenemos disponibles, sus características y las operaciones más frecuentes: copiarlos, moverlos, borrarlos,...

Un módulo en Python es un conjunto de funciones, de código y de funcionalidad en general, que está orientado hacia satisfacer unas necesidades concretas. Por ejemplo existe un módulo *math*, que se encarga de incluir la funcionalidad para trabajar las funciones matemáticas u otro que se llama *pyGame*, que está pensado para ayudar a programar videojuegos.

En un tema posterior veremos en detalles el uso de módulos y cómo utilizarlos. No obstante, vamos a empezar a utilizarlos ya en este, usando los módulos relacionados con ficheros.

Por ahora es suficiente con saber que para trabajar con un módulo sólo tenemos que poner al principio del código la palabra reservada **import** y el nombre del módulo. De esta manera, antes de ejecutar nuestro código, se cargará toda esa funcionalidad contenida en el módulo y la tendremos disponible para utilizarla.

Cuando queramos utilizar una funcionalidad de un módulo concreto, pondremos el nombre del módulo, un punto, y el nombre de esa funcionalidad que vamos a utilizar.

Por ejemplo para usar la función *listdir* del usando módulo *os*, escribiremos *os.listdir()*.

Existen varios módulos que nos permiten trabajar con ficheros. De hecho bastante funcionalidad está replicada en varios de ellos, usaremos uno u otro dependiendo de lo que vayamos a hacer finalmente con los ficheros:

- Si vamos a trabajar solamente revisando dónde están los ficheros pero no vamos a leer su contenido el módulo recomendado es `os`.
- Pero si además de manipularlo lo que queremos es leer o escribir su contenido se recomienda usar el módulo `pathlib`

Operaciones con ficheros

Vamos a ver algunas de operaciones más frecuentes

- Podemos crear directorios con `os.mkdir(directorio)`:

```
import os
os.mkdir('dir') ## Creamos el directorio 'dir'
```

- También podemos crear varios niveles con `os.makedirs(dir1/dir2)`:

```
import os
os.makedirs('nivel1/nivel2/nivel3')
```

- Para borrar un fichero usaremos `remove(fichero)`:

```
import os
os.remove('data.txt') ## Borramos un fichero
```

- Para mover ficheros entre directorios, sólo tenemos que cambiar el nombre usando `rename`. Por ejemplos tenemos varios ficheros en el directorio raíz y queremos moverlos a un directorio 'prueba':

```
import os
os.mkdir('prueba') # crea el directorio '1'
os.rename('irpf_app.py', 'prueba/irpf_app.py') # mueve el fichero dentro del directorio
```

- Si queremos borrar un directorio usaremos `rmdir(dir)`:

```
import os
os.rmdir('1') # borra el directorio
```

- Podemos saber el directorio actual con `os.getcwd()`:

```
import os
directorioActual = os.getcwd()
print(f'Estamos en el directorio {directorioActual})
```

Dentro del módulo `os` existe un grupo de funcionalidad que está pensado para trabajar específicamente con el "path".

Dado que python está pensado para trabajar con diferentes sistemas operativos y algunos de ellos utilizan distinto separador, podemos garantizar que usamos el correcto con `os.path.sep`.

Podemos crear una función "mover", que mueva un fichero creando directorio si es necesario:

```
import os

def mover(fichero, nuevoDirectorio):
    """
    función que hace un move de un fichero
    """
    if nuevoDirectorio not in os.listdir(): # Si no está creado lo creamos
        os.mkdir(nuevoDirectorio)
    if nuevoDirectorio[-1] != os.path.sep: # añadimos la barra
        nuevoDirectorio += os.path.sep
    os.rename(fichero, nuevoDirectorio+fichero)
```

Cuando intentamos hacer estas operaciones puede ocurrir que no tengamos permiso para hacerla, bien por limitaciones de los privilegios del usuario o por los permisos de las carpetas. En ese caso se produce una excepción `PermissionError`

Recorriendo directorios y sus contenidos

Podemos recuperar los ficheros y directorios de un path concreto usando la función `listdir('path')` que nos devuelve una lista con todo el contenido de ese path:

```
import os
for fichero in os.listdir('directorio'):
    print(fichero)
```

A partir del nombre del fichero o directorio podemos recuperar el path absoluto con `os.path.abspath(path)`:

```
import os
for fichero in os.listdir('directorio'):
    print(f'{fichero} -> {os.path.abspath(fichero)}')
```

En el caso contrario, podemos recuperar el nombre del fichero y la carpeta que lo contiene usando `os.path.split(path)`:

```
import os
>>> os.path.abspath('main.py')
'/home/javacasm/Descargas/main.py'
>>> os.path.split(os.path.abspath('main.py'))
('/home/javacasm/Descargas', 'main.py')
```

Podemos recuperar el nombre del fichero de un path con `os.path.basename(path)` y el directorio que lo contiene con `os.path.dirname(path)`:

```
>>> os.path.basename('/home/javacasm/Descargas/main.py')
'main.py'
>>> os.path.dirname('/home/javacasm/Descargas/main.py')
'/home/javacasm/Descargas'
```

En el caso de que no podamos acceder al fichero se producirá una excepción de tipo `FileNotFoundError`.

Información sobre ficheros y directorios

En muchas ocasiones queremos saber si un fichero o directorio existe. Podemos saberlo con `os.path.exists(path)`, que nos devolverá `True` si existe y `False` en caso contrario.

También podemos preguntar si es un fichero o directorio con `os.path.isfile(f)` o `os.path.isdir(f)`:

```
import os
for f in os.listdir():
    if os.path.isfile(f):
        print(f'"{f}" es fichero')
    else :
        print(f'"{f}" es un directorio')
```

Vamos a ver ahora cómo obtener información (por ejemplo el tamaño y las fechas de modificación) sobre un fichero concreto, lo que nos va a mostrar una vez más que en python casi todo se puede hacer de varias formas:

1. Usando el módulo `os.path`

```
import os
import time

nombreFichero = 'fichero.txt'
print('Información del fichero ' + nombreFichero)
print('Tamaño: ',os.path.getsize(nombreFichero))
print('Fecha de creación: ',time.ctime(os.path.getctime(nombreFichero)))
print('Fecha          de          última          modificación: ',time.ctime(os.path.getmtime(nombreFichero)))
print('Fecha de último acceso: ',time.ctime(os.path.getatime(nombreFichero)))
```

Donde hemos usado la función `time.ctime` para convertir el formato de fecha Epoch (segundos desde el 1 de enero de 1970) que se usa en el sistema a un formato más legible.

2. Usando `os.stat()`: Si vamos a recuperar muchos detalles es más eficiente usar la función `stat()` que nos devuelve una estructura con todos los datos:

```
>>> info = os.stat('fichero.txt')
>>> info
os.stat_result(st_mode=33204, st_ino=2345119, st_dev=2051, st_nlink=1,
st_uid=1000, st_gid=1000, st_size=118, st_atime=1612791889,
st_mtime=1577972527, st_ctime=1577972530)
>>> time.ctime(info.st_atime)
'Mon Feb  8 14:44:49 2021'
```

En el caso de que no podamos acceder al fichero se producirá una excepción de tipo `FileNotFoundError`.

Usando `pathlib`

En versiones más recientes de Python, las funciones de manejo de ficheros se han replicado en el módulo `pathlib`, estando casi todas las funciones que hemos comentado en `pathlib.path`. Además de gestionar los ficheros se ha incluido alguna funcionalidad para leerlos o escribir en ellos. Es de suponer que en futuras versiones esta funcionalidad se migre completamente del módulo `os`.

Veamos alguna utilidad de `pathlib`, como es el encontrar los ficheros cuyo nombre cumple un patrón, para lo que podemos usar la función `glob(patron)`:

```
import pathlib

ficherosCodigo = pathlib.Path('.').glob('*.py')
for f in ficherosCodigo:
    print(f'fichero:{f}')
    print(f'nombre:{f.stem}')
    print(f'extensión:{f.suffix}')
    print(f'tamaño:{f.stat().st_size}')
```

Del mismo modo que en el módulo `os` podemos preguntar si existe con `exist()` o si es un fichero con `is_file()` o si es un directorio con `is_dir()`.

Copiar ficheros

Si lo que queremos hacer es copiar ficheros usaremos el módulo `shutils` y sus funciones `copy` o `copy2` según queramos que se copie sólo el fichero o también sus metadatos.

También podemos copiar toda una rama de directorios con sus contenido con `copytree`.

Este módulo también nos permite mover ficheros con `move` y borrar directorios y su contenido con `rmtree`.

Todos estos métodos necesitan que les proporcionemos ficheros o directorio origen y destino.

Exploración recursiva de directorios

Con lo aprendido vamos a hacer un programa que navegue recursivamente un directorio mostrando información del ficheros que encuentre y entrando en todos los subdirectorios para explorarlo

```
import os

listaDirectorios = ['./Descargas']

def showInfo(fichero):
    info = os.stat(fichero)
    print(f'fichero: {fichero} tamaño: {info.st_size}')

def exploraDir(directorio):
    global listaDirectorios
    print('Explorando ' + directorio)
    for fichero in os.listdir(directorio):
        if os.path.isdir(fichero):
            listaDirectorios.append(fichero)
        if os.path.isfile(fichero):
            showInfo(fichero)

while len(listaDirectorios) > 0:
    directorio = listaDirectorios.pop()
    exploraDir(directorio)
```

Lectura de ficheros

Vamos a aprender a leer el contenido de ficheros. Para ello tenemos que empezar abriendo el acceso al fichero con la función `open` a la que pasamos como argumentos el **nombre del fichero** y el **modo** en que queremos abrir el fichero.

Veamos los distintos modos de lectura:

- "r" para lectura.
- "t" para lectura de texto, podremos recuperar línea a línea.
- "b" para lectura de datos binarios.

Por defecto se sobreentiende el modo "rt".

La función *open* nos devuelve una referencia al fichero que usaremos para acceder al mismo

```
f = open('fichero.txt', 'rt')
for linea in f.readlines():
    print(linea)
f.close()
```

Cuando terminemos de usarlo llamaremos a la función *close()* para cerrarlo y liberar recursos.

En el ejemplo anterior veremos que se muestran las líneas separadas en 2 líneas, lo que es debido a que también se lee el carácter de salto de línea "\n" que hay en el fichero y se imprime además el salto de línea de *print*. Podemos evitarlo poniendo el argumento de *print* *end=""* para evitar el salto de línea extra.

Hemos leído todas las líneas del fichero con *readlines()* pero también podíamos haber leído una a una con *readline()* incluso carácter a carácter con *read()* , de N en N con *read(N)*.

En el caso de que no se pueda abrir el fichero se produce una excepción de tipo *FileNotFoundError*.

Codificación de ficheros de texto

Existen diferentes formas de codificar los caracteres. En el inicio de los tiempos de la informática se usaba la codificación **ASCII**, que utilizaba sólo 7 bits, con lo que no permitía codificar más de 128 caracteres ($2^7 = 128$), lo cual era suficiente para trabajar con el idioma inglés, pero dejaba fuera acentos y caracteres de otros lenguajes, por ello se amplió a 8 bits, apareciendo diversas versiones para distintos idiomas. El **ISO-8859-1** o **Latín 1** era la versión correspondiente al español, solucionaba parcialmente el problema porque se usaba una codificación distinta para cada idioma.

Para resolver este problema el **Unicode** que utiliza 32 bits donde ya cabían todos los caracteres de todos los idiomas permitiendo usar 2^{32} (más de 4000 millones) de posibles caracteres.

Para optimizar su uso se creó **Utf-8** (que permite utilizar un tamaño variable de los caracteres y ahorra espacio)

Python 3.x usa Unicode para las cadenas de caracteres, pero cuando abrimos un fichero, para la lectura se usa por defecto la codificación del sistema operativo. Por

todo ello cuando abrimos un fichero de texto y para evitar errores podemos añadir un parámetro de codificación, obligando a que se use una codificación determinada.

Para asegurarnos que se abre con codificación Utf-8 (la más estándar hoy día) añadimos **encoding = "utf8"**

```
...  
file = open(nombre_fichero, "rt", encoding="utf8")  
...
```

Si intentamos leer un fichero con una codificación diferente podemos obtener un error de *"UnicodeDecodeError: 'charmap' codec can't decode byte 0x81 in position 74: character maps to "*, por ejemplo si trabajamos en Windows que usa la codificación **cp1252**.

Escritura de ficheros

Para escribir datos en un fichero, lo abrimos en el modo "w" de escritura.

```
import os  
f = open("myfile.txt", "wt"):  
f.write("Hello world!")  
f.close()  
print(os.listdir()) # comprobamos que se ha creado el fichero
```

El contenido que se escribe es literalmente el que nosotros ponemos explícitamente, con lo que si queremos que se añada un final de línea tendríamos que añadirlo manualmente.

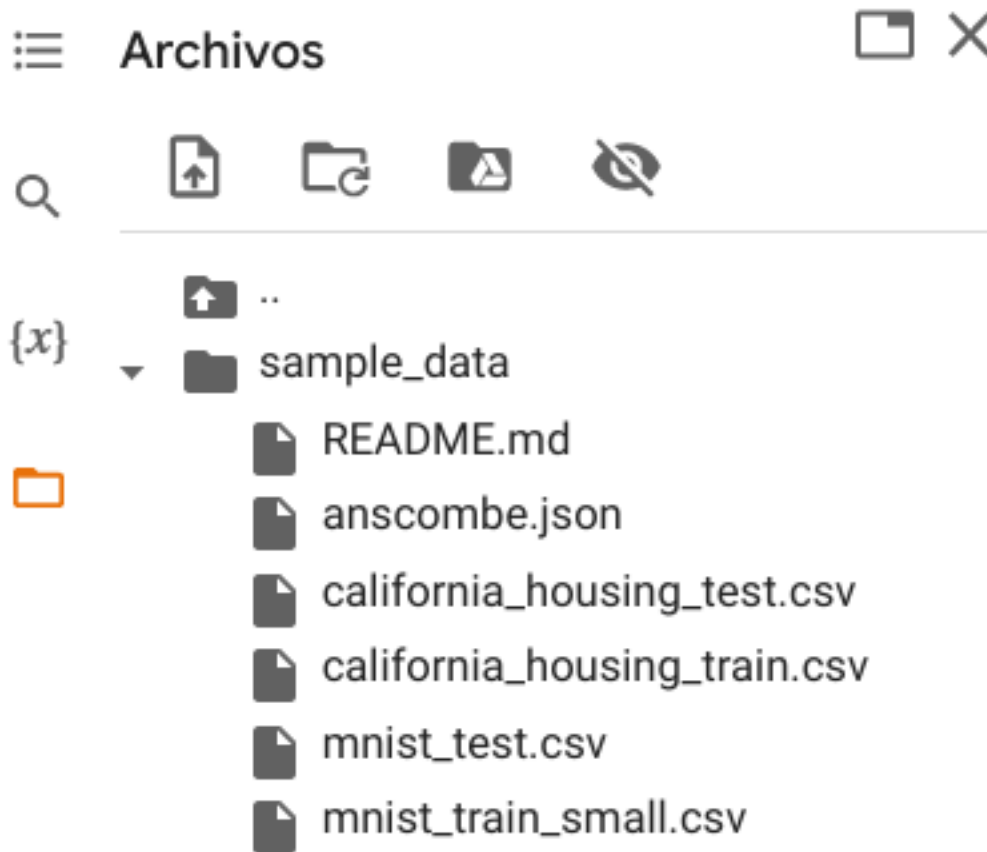
En el caso de que el fichero ya existiera usando el modo "w" éste se sobrescribe. También podemos añadir al contenido al final usando el modo "a".

Si no podemos escribir por permisos o por otra razón se producirá una excepción del tipo *PermissionError*.

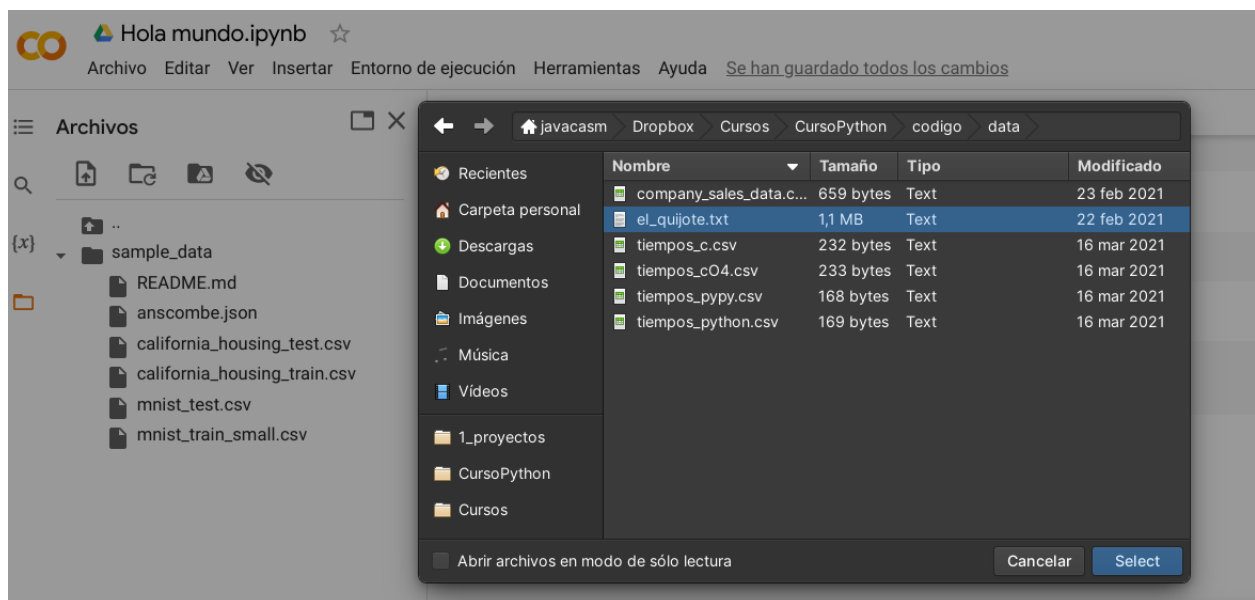
Gestión de ficheros en Google Colab

Vamos a ver cómo podemos incorporar ficheros a nuestro proyecto.

Tenemos un gestor de ficheros sin más que pulsar el icono de "Carpeta" de la barra lateral. Ahí podremos ver las carpetas y ficheros. Por defecto incluye una carpeta de ejemplos (sample_data). Desde ahí podemos añadir, borrar, renombrar, ... nuestros ficheros

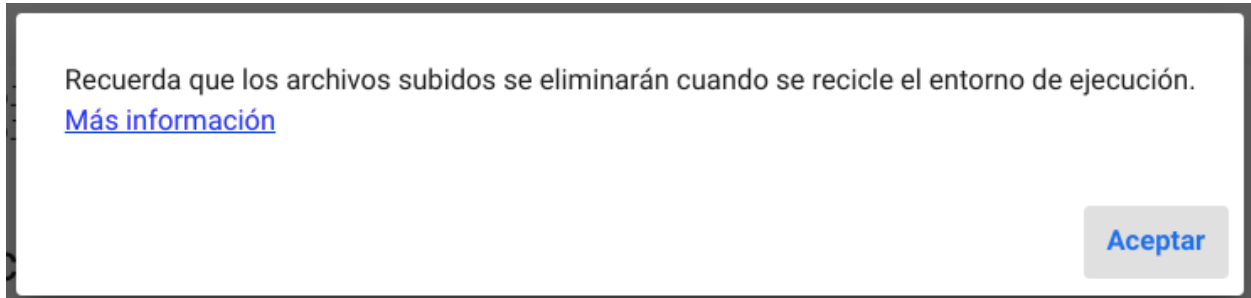


Podemos subir los ficheros desde nuestro ordenador, pulsando el primero icono



También podemos descargar cualquier fichero a nuestro ordenador.

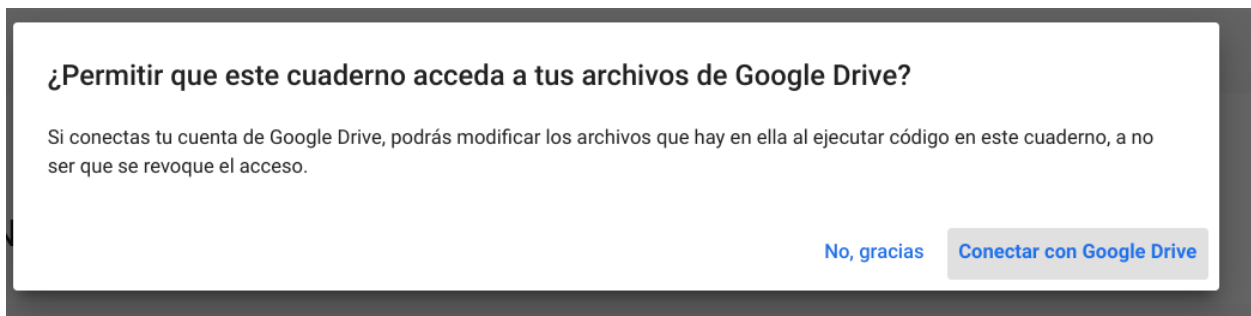
Hay que recalcar que **los ficheros que añadimos a nuestra sesión se pierden cada vez que desconectamos del entorno de ejecución** o lo reciclamos.



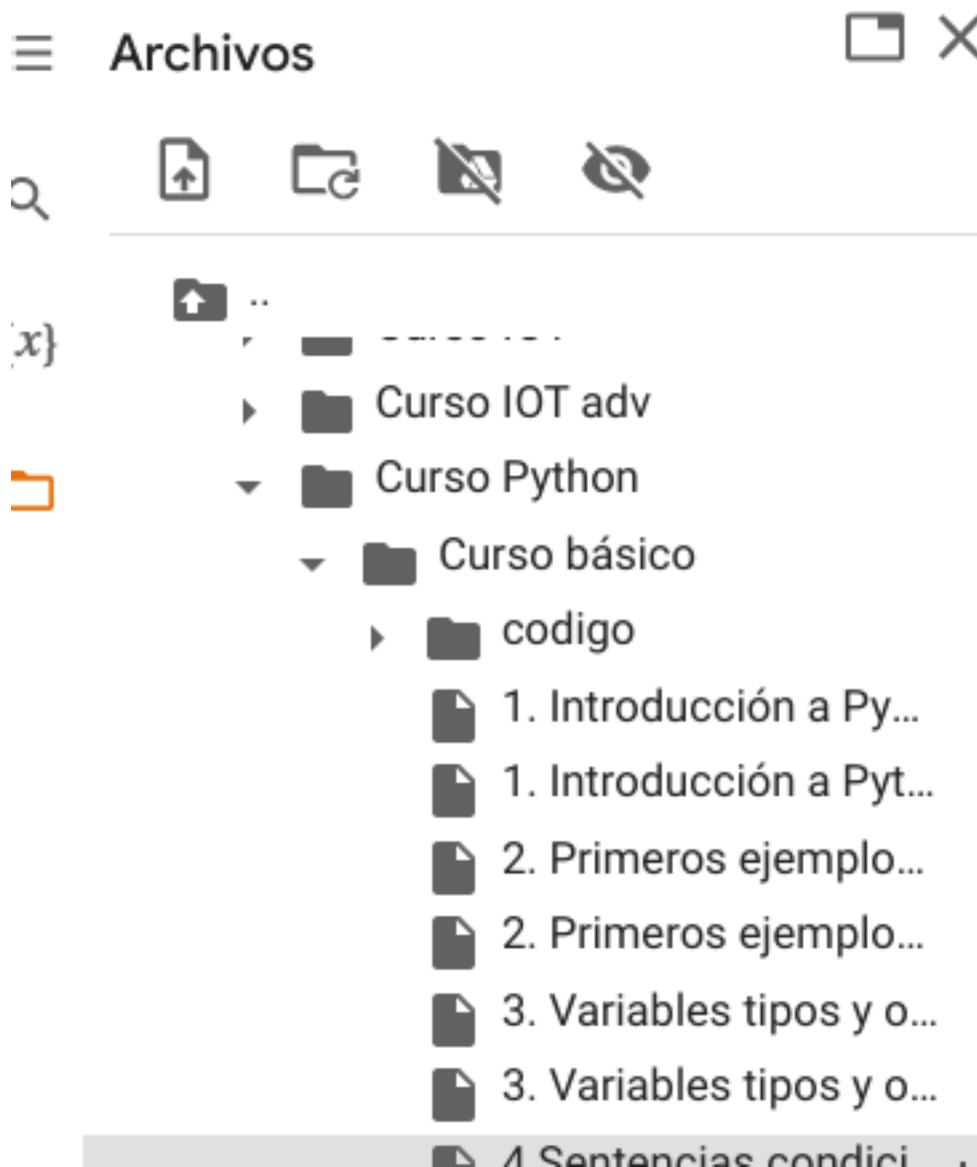
Por esto vamos a ver formas de acceder a documentos en otros almacenamientos como Google Drive o Github.

Conexión con Google Drive

Podemos conectar con Google Drive para acceder a los ficheros allí alojados. Al pulsar sobre el icono de "Google Drive" seleccionaremos nuestra cuenta y nos pedirá permiso para acceder a los ficheros:



Cuando conectamos con Google Drive nos aparecerá una carpeta "drive" entre nuestros ficheros y ahí se mostrarán los ficheros de Drive. Este paso puede tardar si tenemos muchos ficheros:

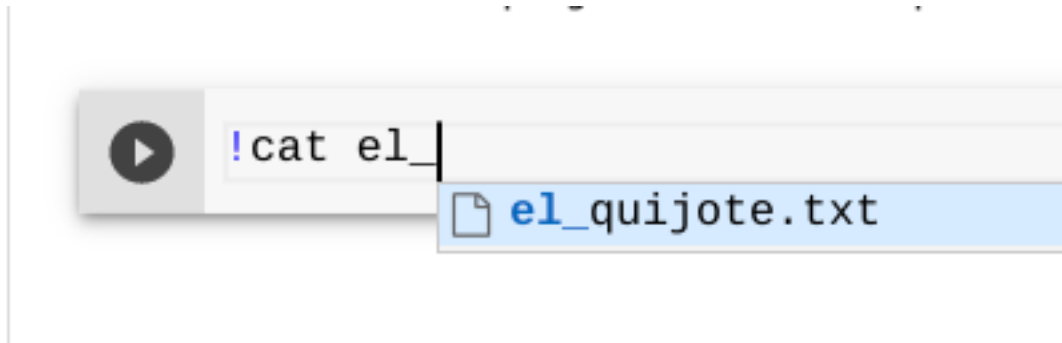


Tras conectar (“montar” en palabras técnicas) con Drive podemos desconectar (desmontar) pulsando en la opción correspondiente.

Comando directos

Podemos ejecutar comandos sobre el servidor al que estamos conectados como haríamos sobre cualquier máquina linux. Para ello anteponeamos una “!” (admiración) al comando por ejemplo para trabajar con ficheros podemos usar:

- **!ls** para listar los ficheros del sistema
- **!cp** para copiar un fichero, por ejemplo “!cp eL_quijote.txt libro1.txt”



- **!rm fichero.txt** para borrar "fichero.txt"
- **!cat fichero.txt** para mostrar el contenido de "fichero.txt"
- **!ps -ef** nos muestra los procesos que se están ejecutando

Conexión con Github

Podemos interaccionar con un repositorio github usando el comando **git clone** como lo haríamos en nuestro ordenador.

- **!git clone repositorio.git**

